

Architecture Proposal: WhatsApp Notification Engine

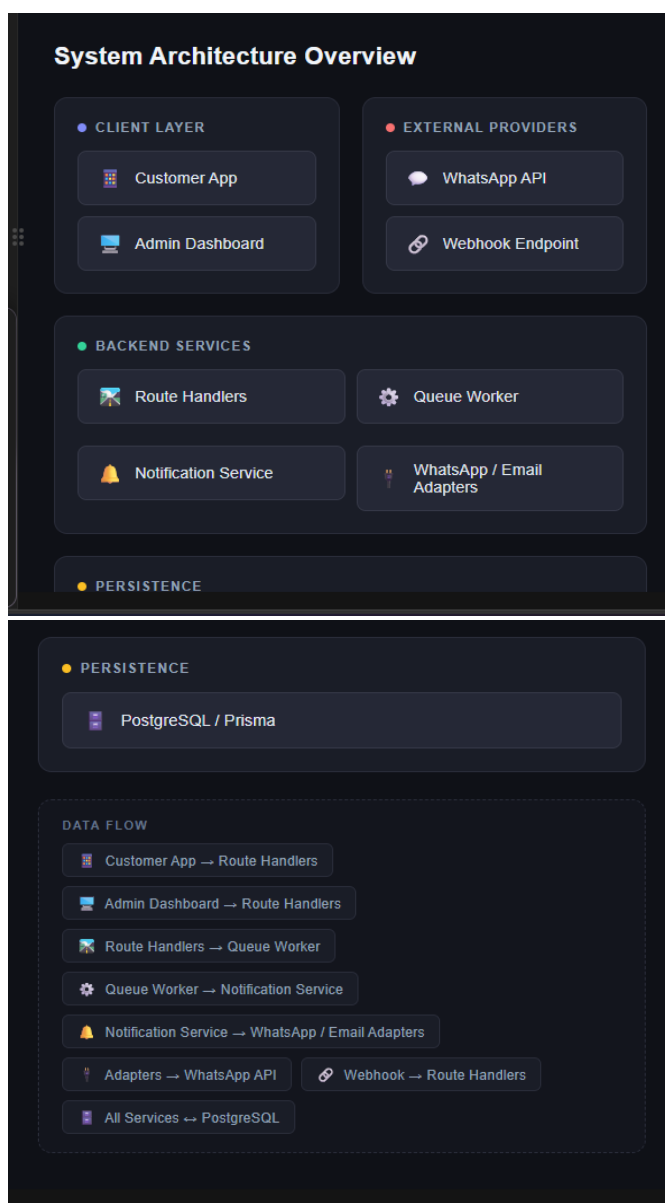
Objective: Replace SMTP-based notifications with a resilient, auditable WhatsApp pipeline integrated into the loan lifecycle.

1. System Evolution (As-Is vs. To-Be)

- **Current State:** Tight coupling between business logic and email.ts. Notifications are sent synchronously, creating risk for API timeouts.
- **Target State:** Decoupled architecture using a **Notification Outbox**. Business logic only writes to the DB; a background worker handles the external I/O.

2. Proposed Architecture

2.1 Component Diagram



3. Data Engineering

3.1 Schema Additions (Prisma)

We will introduce a NotificationDelivery model to act as our system of record.

Field	Type	Description
id	UUID	Primary Key
status	Enum	QUEUED, SENT, DELIVERED, READ, FAILED
channel	Enum	WHATSAPP, EMAIL
payload	JSON	Template variables (e.g., { "customerName": "John" })
providerId	String?	Remote ID from Meta/Twilio for tracking
attempts	Int	Counter for exponential backoff logic
retryAt	DateTime	Schedule for next attempt after failure

3.2 User Profile Extensions

- whatsappNumber: Normalized **E.164** format.
- whatsappOptInAt: Timestamp for legal compliance.
- communicationPrefs: User-defined priority (WhatsApp vs. Email).

4. Implementation Strategy

4.1 The Outbox/Worker Pattern

To ensure the backend remains fast and reliable, we will avoid "inline" sending:

1. **Enqueuing:** When a loan is approved, admin.ts creates a NotificationDelivery record within the same DB transaction.
2. **Processing:** A background worker (extending existing logic in jobs/loanReminders.ts) picks up QUEUED records.
3. **Idempotency:** Every event maps to a unique loanId + eventType key to prevent duplicate messages.

4.2 Integration Interface

We will implement a NotificationProvider interface to prevent vendor lock-in:

TypeScript

```
interface NotificationProvider {  
  send(to: string, templateId: string, vars: object): Promise<ProviderResponse>;  
  handleWebhook(payload: any): StatusUpdate;  
}
```

5. Security & Compliance

- **Signature Verification:** The Webhook endpoint must validate the X-Hub-Signature (Meta) or X-Twilio-Signature to prevent spoofing.
- **PII Masking:** Templates will use references (e.g., "Your loan ending in *4421") rather than full sensitive details.
- **Opt-Out Logic:** Automated handling of "STOP" keywords to immediately flag `User.whatsappEnabled = false`.

6. Operational Requirements

- **Reliability:** Initial send within < 60s of trigger.
- **Backoff Policy:** Retries at 1m, 5m, 15m, 1h, and 6h before moving to a Dead Letter Queue (DLQ).
- **Monitoring:** Admin dashboard to show real-time "Read Receipts" and failure reasons (e.g., "Invalid Number").

7. Immediate Sign-off Needed

1. **Provider Choice:** Meta Cloud API (direct) vs. Twilio (managed).
2. **Fallback Strategy:** We auto-send an Email if the WhatsApp delivery fails after 3 retries
3. **OTP Migration:** User registration/verification move to Whatsapp
4. **Verification Token:** Approval of the WHATSAPP_WEBHOOK_VERIFY_TOKEN security protocol.